DAY 04 OF 30

# Apache Airflow & Pipeline Orchestration

Configure Managed Airflow, design DAGs, choose the right tool, and master triggers for the DP-700 exam

- Now in DP-700 Syllabus
- Managed Airflow in Fabric
- DAG Architecture
- Schedule vs Event Triggers
- 3 Practice Questions

Authored by Sayan Banerjee • Microsoft Certified: Fabric Data Engineer Associate

OFFICIAL EXAM UPDATE → DP-700 Study Guide • July 21, 2026

# Apache Airflow is Now on the DP-700 Exam ✅

We verified this live from the official Microsoft Learn study guide — this is NOT optional knowledge

📋 **Official Exam Objective (July 21, 2026)**

"Configure Microsoft Fabric workspace settings" →
**"Configure Apache Airflow workspace settings"**

Domain: Implement and manage an analytics solution (30–35%)

### 🎯 Why It Matters

30–35% of the exam is "Implement & Manage". Airflow workspace config is directly in this domain — high probability question.

### 🗓️ When Added

Added in the July 21, 2026 revision as a **Minor change** to workspace settings configuration skills.

### 🔗 Also Covers

Orchestrate processes → Choose Pipeline vs Notebook vs Airflow → Design triggers → Parameters & dynamic expressions.

ORCHESTRATE PROCESSES → Choose the right tool for the job

# 4 Orchestration Tools in Fabric

The DP-700 tests your ability to choose the correct tool for each scenario

### Data Pipeline

GUI LOW-CODE

- Multi-activity chains
- Copy, notebook, SP, web
- Schedule + event triggers
- Parameters & expressions
- Retry & error routing

Best for ELT flows

### Notebook

PYSPARK / SQL / KQL

- PySpark at Spark scale
- Called via Pipeline
- Cannot self-schedule ⚠
- mssparkutils for control
- Interactive + scheduled

Best for compute

### Dataflow Gen2

POWER QUERY M

- No-code drag & drop
- Power Query M engine
- Built-in scheduler
- DirectQuery sources
- NOT for PySpark logic

Best for M transforms

### Apache Airflow

MANAGED / DAG-BASED

- Python DAG definitions
- Complex dependencies
- Multi-system operators
- Fabric-native operators
- Configured in Workspace

Best for complex DAGs

WORKSPACE SETTINGS → Configure Apache Airflow (Exam Objective)

# Managed Airflow: Workspace Configuration

⚠ **Exam Key Fact**

Apache Airflow is configured at the WORKSPACE level → Workspace Settings → Apache Airflow — NOT at tenant level or item level.

| Workspace Setting | What It Controls | Notes |
|---|---|---|
| **Enable Airflow** | Activates managed Airflow in the workspace | Toggle on/off |
| **Environment Size** | Small / Medium / Large compute allocation | Cost driver |
| **DAG Folder (OneLake path)** | Where Python DAG .py files are stored | Native OneLake integration |
| **Plugin Folder** | Custom Airflow operators and plugins | Optional |
| **Requirements.txt** | Extra Python packages for DAG dependencies | pip install on startup |
| **Config Overrides** | Key-value pairs overriding airflow.cfg defaults | e.g. parallelism, max_active_runs |

AIRFLOW DAGs → Directed Acyclic Graph — the core orchestration unit

# DAGs + Fabric-Native Operators

**What is a DAG?**

A Python script defining Tasks + Dependencies with no circular paths. "Acyclic" = no loops allowed.

```python
from airflow import DAG
from airflow.providers.microsoft.fabric.operators.notebook import FabricRunNotebookOperator

with DAG(
  dag_id='dp700_bronze_to_silver',
  schedule_interval='0 6 * * *', # Daily 06:00
  catchup=False
) as dag:
  ingest = FabricRunNotebookOperator(
    task_id='ingest_raw_data',
    notebook_id='<id>'
  )
  transform = FabricRunNotebookOperator(
    task_id='transform_bronze',
    notebook_id='<id>'
  )
  ingest >> transform # dependency arrow
```

## Fabric-Native Airflow Operators

**FabricRunNotebookOperator**

Triggers a Fabric Notebook by workspace_id + notebook_id. Supports parameters.

**FabricRunPipelineOperator**

Triggers a Data Factory Pipeline in Fabric. Waits for completion by default.

**FabricRunDataflowOperator**

Refreshes a Dataflow Gen2. Useful for chaining Dataflows with other Fabric workloads.

ORCHESTRATE PROCESSES → Design and implement schedules and event-based triggers

# Schedule vs Event-Based Triggers

| Trigger Type | Tool | When to Use | Exam Notes |
|---|---|---|---|
| ⏱ **Schedule Trigger** | Pipeline, Dataflow Gen2, Airflow DAG | Fixed time intervals — daily, hourly, CRON | Notebook CANNOT self-schedule |
| 📁 **Storage Event Trigger** | Pipeline | File arrives/modified in OneLake or ADLS Gen2 | Most common event trigger in DP-700 |
| 🟦 **Tumbling Window** | Pipeline | Time-sliced micro-batches with no overlap | Distinct from schedule — has "window" concept |
| 🔔 **Fabric Activator (Reflex)** | Activator item | Metric threshold / data condition met | Not a Pipeline trigger — separate Fabric item |
| 🌀 **Airflow Sensor** | Airflow DAG | Wait for external condition (file, task, API) | ExternalTaskSensor, FileSensor, HttpSensor |

💡 **Key DP-700 Distinction**

Schedule trigger = time-driven. Storage event trigger = data-driven. Tumbling window = time-sliced batches. Activator = metric-driven. These are all different trigger types — the exam may ask you to pick the right one.

ORCHESTRATION → Parameters, dynamic expressions, and control flow patterns

# Orchestration Patterns & Parameters

### ➡️ Sequential Pattern

Ingest → Transform → Load

Default pipeline chain. Activity A must succeed before B starts.

### Parallel Pattern (ForEach)

Use `ForEach` with **isSequential: false** to process multiple items concurrently. Example: ingest 5 tables in parallel.

### 🔀 Conditional Pattern (If)

Use `If Condition` activity. Expression evaluates to true/false → routes to different activity branches.

### 📐 Dynamic Parameters & Expressions

```
// Pipeline parameter — dynamic date
{
  "load_date": "@formatDateTime(utcNow(), 'yyyy-MM-dd')",
  "env": "production"
}

# Notebook receives parameters via:
mssparkutils.notebook.run(
  "Silver_Transform", 600,
  {"load_date": "2026-07-21"}
)

# Airflow Jinja template equivalent:
execution_date = "{{ ds }}" # run date
```

### Retry & Error Handling

Each pipeline activity has: Retry count, Retry interval (seconds), On Failure / On Skip / On Success paths for routing.

DP-700 CAUTION → Orchestration questions that trick candidates

# Top 4 Orchestration Exam Traps ⚠️

**Trap 1: "Schedule a Notebook directly"**

❌ WRONG. Notebooks CANNOT self-schedule. You must wrap them in a Pipeline with a Schedule Trigger, or call them via an Airflow DAG.

**Trap 2: "Configure Airflow at tenant level"**

❌ WRONG. Airflow is configured at WORKSPACE level: Workspace Settings → Apache Airflow. Not Admin Portal, not item level.

**Trap 3: "Use Dataflow Gen2 for PySpark logic"**

❌ WRONG. Dataflow Gen2 uses Power Query M — no PySpark. For PySpark at scale, use a Notebook called from a Pipeline.

**Trap 4: "Tumbling Window = Schedule Trigger"**

❌ WRONG. Tumbling Window is time-sliced with non-overlapping windows and backfill support. Schedule trigger is a simple recurrence. They solve different problems.

QUICK REFERENCE → When to use which orchestration tool

# Orchestration Decision Matrix

| Scenario | Best Tool | Why |
|---|---|---|
| Simple ELT: Copy data → Transform notebook → Write to Lakehouse | Data Pipeline | Native Fabric-to-Fabric GUI, schedule trigger, sequential activities |
| Complex cross-system DAG with 20+ tasks and conditional branches | Apache Airflow | Full DAG dependencies, Python operators, multi-system support |
| Non-technical team needs to transform data without code | Dataflow Gen2 | Power Query M, no-code UI, direct semantic model refresh |
| Run PySpark transformations at large scale | Notebook (via Pipeline) | Notebook runs on Spark; Pipeline schedules and passes parameters |
| Trigger pipeline when CSV file arrives in OneLake folder | Pipeline Storage Event Trigger | Event-driven — fires on file arrival/modification |
| Alert and trigger action when sales metric drops below threshold | Fabric Activator (Reflex) | Metric/data condition monitoring, not a pipeline trigger |

# Now you know Airflow.
# Most DP-700 candidates don't.

Follow the 30-Day DP-700 challenge to master every exam domain with real-world context, career-ready visuals, and practice questions.

**🔔 Day 05 Tomorrow: Ingest & Transform Data — Pipelines Deep Dive**

**Sayan Banerjee** • AI, BI & Analytics Consultant
Microsoft Certified: Fabric Data Engineer Associate (DP-700)